

Graph AI Engine - End-to-End Implementation - Esakiraj Serman

1. Project Structure

```
graph-ai-engine/  
|  
├─ app/  
|   ├── main.py  
|   ├── config.py  
|   ├── graph_engine.py  
|   ├── schemas.py  
|   └── utils.py  
|  
├─ .env  
├─ requirements.txt  
└─ README.md
```

2. requirements.txt

```
fastapi  
uvicorn  
neo4j  
python-dotenv  
langchain-core  
langchain-openai  
langchain-neo4j
```

3. .env

```
NEO4J_URI=bolt://localhost:7687  
NEO4J_USER=neo4j  
NEO4J_PASSWORD=password123  
  
OPENAI_API_KEY=your_key
```

```
OPENAI_MODEL=gpt-4o-mini
BASE_URL=https://.openai.com/v1
```

4. app/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Settings:
    NEO4J_URI = os.getenv("NEO4J_URI")
    NEO4J_USER = os.getenv("NEO4J_USER")
    NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")

    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
    OPENAI_MODEL = os.getenv("OPENAI_MODEL")
    BASE_URL = os.getenv("BASE_URL")

settings = Settings()
```

5. app/utils.py

```
FORBIDDEN = ["DELETE", "CREATE", "MERGE", "SET", "DROP", "REMOVE", "CALL dbms"]

def clean_cypher(raw: str) -> str:
    if "```" in raw:
        raw = raw.split("```")[1]
        raw = raw.replace("cypher", "").strip()
    return raw.strip()

def validate_cypher(query: str) -> str:
    upper = query.upper()
    for word in FORBIDDEN:
        if word in upper:
            raise Exception("Blocked dangerous query.")
    if "MATCH" not in upper:
        raise Exception("Only MATCH read queries allowed.")
    return query

def normalize_result(result):
```

```

if not result:
    return {"type": "empty"}

if len(result) == 1 and len(result[0]) == 1:
    return {"type": "scalar", "value": list(result[0].values())[0]}

return {"type": "table", "rows": result}

```

6. app/graph_engine.py

```

from langchain_neo4j import Neo4jGraph
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from .config import settings
from .utils import clean_cypher, validate_cypher, normalize_result

# Neo4j Graph
graph = Neo4jGraph(
    url=settings.NEO4J_URI,
    username=settings.NEO4J_USER,
    password=settings.NEO4J_PASSWORD,
    refresh_schema=True
)

SCHEMA = graph.schema

# LLM
llm = ChatOpenAI(
    api_key=settings.OPENAI_API_KEY,
    base_url=settings.BASE_URL,
    model_name=settings.OPENAI_MODEL,
    temperature=0.1,
)

# Cypher Generator
cypher_prompt = ChatPromptTemplate.from_template("""
You are an expert Neo4j Cypher engineer.

GRAPH SCHEMA:
{schema}

Rules:
- Use only schema elements.

```

- Use case-insensitive matching with toLower().
- Prefer CONTAINS for flexible matching.
- Only MATCH queries allowed.
- Never generate DELETE, CREATE, MERGE, SET, DROP.
- Output ONLY Cypher.

User Question:
{question}

Cypher:
""")

cypher_chain = cypher_prompt | llm | StrOutputParser()

Formatter

answer_prompt = ChatPromptTemplate.from_template("""
User Question:
{question}

Database Result:
{result}

Explain clearly and concisely.
If empty → say no records found.
If scalar → short answer.
If table → bullet list.
""")

formatter_chain = answer_prompt | llm | StrOutputParser()

```
def run_graph_query(question: str):
    current_question = question

    for _ in range(2):
        raw_cypher = cypher_chain.invoke({"schema": SCHEMA, "question":
current_question})
        cypher = clean_cypher(raw_cypher)
        cypher = validate_cypher(cypher)

        if "LIMIT" not in cypher.upper():
            cypher += "\nLIMIT 100"

    try:
        result = graph.query(cypher)
        normalized = normalize_result(result)
```

```

        if normalized["type"] == "empty":
            return "No matching records found."

        if normalized["type"] == "scalar":
            return f"The answer is: {normalized['value']}"

        return formatter_chain.invoke({"question": question, "result":
normalized["rows"]})

    except Exception as e:
        current_question = f"""
Previous query failed with error:
{str(e)}

Regenerate correct Cypher.
Original question:
{question}
"""

        raise Exception("Query failed after retry.")

```

7. app/schemas.py

```

from pydantic import BaseModel

class QueryRequest(BaseModel):
    question: str

class QueryResponse(BaseModel):
    answer: str

```

8. app/main.py

```

from fastapi import FastAPI, HTTPException
from .graph_engine import run_graph_query
from .schemas import QueryRequest, QueryResponse

app = FastAPI(
    title="Space Mission Graph AI Engine",
    version="1.0.0",
    description="Production-ready Universal Graph QA System for Space Missions"
)

```

```

)

@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.post("/query", response_model=QueryResponse)
def query_graph(request: QueryRequest):
    try:
        answer = run_graph_query(request.question)
        return QueryResponse(answer=answer)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

9. Run Locally

```
uvicorn app.main:app --reload
```

Swagger UI: <http://127.0.0.1:8000/docs>

Test example space mission queries:

```

run_graph_query("How many astronauts are currently on active missions?")
run_graph_query("List spacecraft where astronauts with Python skill (space
software) are assigned?")
run_graph_query("Which space agency has most active missions?")
run_graph_query("How many open mission tasks? and give me the list of those
tasks?")
run_graph_query("Find astronauts onboarded count between 2022-01-01 and
2025-01-01")
run_graph_query("Find all available astronauts with skill X and match them with
open missions requiring that skill")

```

10. Portfolio Highlights

- LLM → Neo4j Cypher generation for space mission data
- Schema grounding
- Query guardrails
- Retry logic
- LLM-based result formatting
- FastAPI deployment

- Health endpoint
- Modular and production-ready
- Ready for cloud deployment

This is a **complete production-grade Space Mission Graph AI system.**